COMP6211J Course Report

Zhenghong Huang
zhuangdr@connect.ust.hk

Abstract

The exponential growth of Large Language Models (LLMs) has increased the demand for high-end hardware, making it increasingly common to rely on heterogeneous clusters (e.g., mixing Nvidia H100s with older V100s) for training Mixture-of-Experts (MoE) models. While recent systems such as HeterMoE enable training on such clusters via static layer disaggregation, they operate at a relatively coarse micro-batch granularity that can be vulnerable to workload skew—a common phenomenon where specific experts receive disproportionate traffic. Conversely, systems such as Comet achieve fine-grained computation-communication overlap, but their rigid dependency assumptions make them less robust on heterogeneous hardware. This paper proposes **HeteFlow**, a unified MoE training system that bridges this gap through three contributions: (1) We formulate the MoE routing problem as an *Online Stochastic Resource-Constrained Project Scheduling Problem (SRCPSP)*, solving it via a lightweight greedy heuristic; (2) We introduce *Wave-Based Elastic Scheduling*, leveraging insights from FlashOverlap to optimize PCIe latency by grouping tokens into waves rather than individual tiles; (3) We implement a *Polymorphic Dispatch Kernel* using the ParallelKittens framework, enabling dynamic "Shadow Expert" execution to offload stragglers. We hypothesize that HeteFlow can maintain high training throughput even under extreme skew (e.g., Zipfian $\alpha > 1.2$), and aim to show that it outperforms static heterogeneous baselines by reducing pipeline stalls.

1 Literature Review: MoE Systems

The trajectory of Mixture-of-Experts (MoE) research has shifted from an initial focus on model architecture to an intense period of systems-level optimization. This evolution has been driven by the physical constraints of hardware: first memory capacity, then compute utilization, and finally, the latency of inter-device communication in increasingly heterogeneous environments.

1.1 Foundations and the Scale-Out Era

The theoretical basis of conditional computation was introduced by Jacobs et al., who proposed *Adaptive Mixtures of Local Experts* [8]. Their key idea was to decompose a complex problem space into sub-regions, each handled by a specialized neural network ("expert") under the control of a gating mechanism. This formulation anticipated modern MoE architectures by framing learning as both specialization (experts) and arbitration (gating). However, it saw limited uptake in early deep learning, in part because dense scaling remained feasible and hardware constraints did not strongly incentivize sparsity.

The resurgence came with Shazeer et al.'s *Sparsely-Gated MoE Layer* [12], motivated by the escalating cost of LSTM language models. They introduced Noisy Top-K Gating, which injects Gaussian noise into routing logits to encourage exploration and mitigate the "expert collapse" failure mode, where the model degenerates to using only a few experts. This work helped make MoEs practical at scale and reframed sparsity as a tool for increasing capacity without proportionally increasing computation.

Preprint. Under review.

Google's *GShard* (2020) [10] marked the transition from single-machine experiments to massive distributed MoEs, scaling to 600B parameters. Constrained by TPU memory and XLA's static graph compilation, GShard automatically sharded experts across TPU pods and established the now-standard Top-2 gating. To preserve strict synchrony across thousands of devices, it introduced a fixed "capacity factor," i.e., a hard per-expert token budget. Tokens routed beyond this capacity were dropped outright. This design ensured predictable memory use and stable training, but at the cost of dropped tokens and potentially suboptimal expert utilization, highlighting a central tension between hardware efficiency and expert specialization that influenced subsequent work.

1.2 The Commodity Era: Solving the Dropped Token Problem

As MoEs moved from tightly coupled TPU pods to more diverse GPU clusters, static capacity constraints became increasingly restrictive. *FastMoE* (2021) [5] addressed this by embracing dynamic execution. Built on PyTorch rather than XLA, it used flexible CUDA stream management to support dynamic token routing and variable batch sizes, substantially lowering the barrier to MoE experimentation outside large industrial labs.

Dynamic routing, however, amplified the cost of the All-to-All communication required for distributed MoE: tokens must be sent from their originating GPU to the GPUs hosting the selected experts and then gathered back. Microsoft's *Tutel* (2022) [7] attacked this bottleneck at the communication layer. It introduced adaptive parallelism, dynamically switching between hierarchical All-to-All (aggregating via NVLink within a node before using InfiniBand) and direct dispatch based on runtime profiling. Tutel highlighted that, in many settings, network bandwidth rather than compute FLOPs can become the key determinant of MoE throughput.

MegaBlocks (2022) [3] delivered a further breakthrough by rethinking how expert workloads are represented. Prior systems relied on padded, rectangular tensors to satisfy dense GEMM kernels, causing memory waste and "compute bubbles" when expert loads were skewed. MegaBlocks proposed *Block-Sparse Matrix Multiplication (BSMM)*, a family of kernels that operate directly on jagged, variable-sized expert batches. By eliminating both padding and token dropping in its design, it showed that systems can more efficiently accommodate the naturally skewed (often Zipfian) distribution of expert activations. This insight suggests that, rather than enforcing uniform load via strong auxiliary losses—which may affect model quality—it can be beneficial to engineer systems that remain efficient under intrinsic imbalance.

1.3 The 2025 Frontier: Fine-Grained Overlapping and Heterogeneity

Recently, the continued growth of model sizes combined with limited availability of high-end GPUs (e.g., Nvidia H100s) has led to a research landscape that increasingly differentiates between homogeneous and heterogeneous deployment scenarios. One line of work focuses on maximizing performance on homogeneous clusters through aggressive pipelining, while another explores enabling efficient training on heterogeneous clusters composed of mixed hardware generations.

Homogeneous Optimization and the Granularity Mismatch. Zhang et al. introduced *Comet* [16] to address several inefficiencies of standard micro-batch pipelining. Comet identifies a "granularity mismatch" in previous systems: while tokens are routed individually, high-performance GEMM kernels require large tiles. Standard approaches, like Megatron-LM, force a GPU to wait for the entire All-to-All communication phase to finish before beginning computation. Comet introduced a "Shared Tensor" mechanism that analyzes data dependencies at the tile level. By fusing communication and computation kernels, Comet allows a GPU to begin computing on the first arriving tile of tokens while subsequent tiles are still in transit. This fine-grained overlapping achieved up to a $1.71\times$ speedup in their experiments, but also introduced a potential fragility: it assumes relatively deterministic execution times across devices. Comet's dependency graph is relatively rigid: if a single GPU in the cluster is noticeably slower than the others—a "straggler"—the tightly coupled pipeline can stall, and the overlap benefits may be significantly reduced. Thus, Comet is particularly effective for homogeneous clusters, but its assumptions make it less well suited to environments with pronounced hardware heterogeneity.

Heterogeneity and Static Disaggregation. To better utilize available hardware, Wu et al. proposed *HeterMoE* [15]. They leveraged the observation that newer GPUs (such as the H100) provide substantial speedups for compute-bound operations like Attention (due to Tensor Core advancements),

but offer smaller relative gains for memory-bound operations like Feed-Forward Networks (Experts) compared to older generations (such as the V100). HeterMoE introduced "Zebra Parallelism," a strategy that statically disaggregates the model across devices: Attention layers are placed on newer GPUs, and Expert layers are placed on older GPUs. Execution zig-zags between them, overlapping the heavy compute of the H100 with the bandwidth-intensive dispatch of the V100.

However, HeterMoE relies on an "Asymmetric Expert Assignment" strategy that is static. Experts are permanently assigned to specific devices. This design is fatal in the presence of dynamic workload skew. In real-world language modeling, token routing can be highly bursty; a single prompt may trigger a large surge of traffic to a specific set of experts. If those "hot" experts happen to reside on a slower V100, that device can become a bottleneck for the entire cluster, while the faster H100s may experience increased idle time waiting for the straggler to finish. Under such conditions, HeterMoE's coarse, micro-batch granularity provides limited flexibility to dynamically offload the surge.

1.4 Engineering Primitives for Dynamic Scheduling

To reconcile Comet's emphasis on fine granularity with HeterMoE's focus on robustness in heterogeneous settings, it is useful to consider recent low-level engineering advances. *FlashOverlap* (2025) *FlashOverlap* [6] revisits the token-level granularity used in early overlapping attempts. It shows that on bandwidth-constrained interconnects—such as PCIe links connecting different GPU generations in heterogeneous clusters—sending very small tiles can significantly reduce throughput due to protocol overhead. Instead, it proposes grouping tokens into "Waves" to improve bus utilization, and provides a lower bound on efficient scheduling granularity under such constraints.

In parallel, *ParallelKittens* [13] exposes hardware primitives that simplify the implementation of complex, self-managing kernels. By providing abstractions for the Tensor Memory Accelerator (TMA) and register-level communication, ParallelKittens facilitates the construction of "Polymorphic Kernels" that can manage their own work queues and communication logic with minimal CPU intervention. Additionally, *DeepEP* (2025) [2] provides an optimized communication library specifically for Expert Parallelism, serving as a baseline for intrinsic-based dispatch.

HeteFlow is designed to synthesize these diverse advancements. It adopts the heterogeneous architecture of HeterMoE to better utilize available hardware and borrows the overlapping philosophy of Comet to improve throughput. However, it replaces static assignment with a dynamic, operations-research-driven scheduler. By leveraging the wave-based granularity of FlashOverlap and the kernel primitives of ParallelKittens, HeteFlow implements an online "Shadow Expert" mechanism that dynamically offloads straggling work from slow GPUs to fast ones, aiming to mitigate the skew-related vulnerabilities observed in current state-of-the-art systems.

2 Literature Review: Operations Research Foundations

To address the scheduling complexities inherent in heterogeneous Mixture-of-Experts (MoE) training, we complement heuristic systems approaches with insights from foundational Operations Research (OR). The problem of mapping a data-dependent neural network workload onto an asymmetric cluster can be formalized through the lens of Resource-Constrained Project Scheduling, Stochastic Optimization, and Online Algorithms.

Deterministic Scheduling and Its Limits. The execution of a distributed neural network is classically modeled as a Directed Acyclic Graph (DAG), where nodes represent computation tasks (e.g., GEMM kernels) and edges represent data dependencies (e.g., tensor transfers). The problem of mapping such a DAG onto a set of heterogeneous processors to minimize the makespan is commonly referred to as the Heterogeneous DAG Scheduling problem. Topcuoglu et al. (2002) [14] introduced the *Heterogeneous Earliest Finish Time (HEFT)* algorithm, which remains a widely used baseline. HEFT computes a static "upward rank" for each task based on the average communication and computation costs, greedily assigning critical-path tasks to the fastest available processors.

While HEFT provides a useful framework for static graphs, it relies on the assumption that task durations are deterministic and known *a priori*. In the context of MoE, this assumption is often violated. The "duration" of an expert computation is a function of the number of tokens routed to it, a value that is stochastic, governed by the gating network, and highly variable due to Zipfian workload skew. A static schedule derived via HEFT can be brittle in this setting; a single "bursty" batch routed

to a slow processor (e.g., a V100) may significantly degrade performance, causing pipeline stalls that deterministic models cannot anticipate.

Stochastic Scheduling and Queuing Theory. To account for the variability in token routing, the problem can be framed as a *Stochastic Resource-Constrained Project Scheduling Problem (SRCPSP)*. Pinedo (2016) [11] provides foundational theory for scheduling tasks with random processing times. In this regime, the objective often shifts from minimizing the absolute makespan to minimizing the *expected* makespan, possibly subject to variance or tail-latency constraints.

A key insight from this literature is the application of queuing theory to heterogeneous server farms. In such models, the system can be represented as a collection of fast servers (G_{fast}) and slow servers (G_{slow}) with distinct service rates μ. When the arrival rate of tasks (token waves) targeting a specific logical expert exceeds the service rate of its assigned physical processor (a straggler scenario), the corresponding queue can grow unbounded. Classical policies suggests that static assignment is suboptimal in high-variance regimes. Instead, flexible assignment strategies, where tasks can be routed to any capable server, often yield significantly lower tail latencies. This observation motivates HeteFlow's use of a dynamic routing policy instead of purely static assignment as in HeterMoE.

Work Stealing and Dynamic Load Balancing. Since exact solutions to Stochastic RCPSP are computationally intractable for real-time systems, we look to dynamic approximations. Blumofe and Leiserson (1999) [1] analyzed *Work Stealing* schedulers for dynamic DAGs and established strong theoretical performance guarantees. Their "Cilk" runtime demonstrated that a distributed system where idle processors "steal" work from busy processors can, under certain assumptions, achieve execution times within a constant factor of the optimal offline schedule.

HeteFlow adopts a hardware-aware variation of work stealing in its design. In our model, the fast H100 GPUs act as the "thieves." When the queue depth of a slow V100 exceeds a threshold derived from Little's Law—indicating that the latency of waiting for the V100 is expected to exceed the cost of local execution—the H100 "steals" the token wave. However, unlike standard work stealing where code moves to data, HeteFlow must move data (expert weights) to the compute. This leads to a specific class of scheduling problem where the setup cost (transferring "Shadow Expert" weights) is non-zero but predictable, enabling an explicit cost-benefit analysis before stealing occurs.

The Online Knapsack Problem. Finally, the decision of whether to offload a specific group of tokens ("Wave") to a slow GPU is constrained by a limited resource: the available PCIe bandwidth. This can be related to the *Online Knapsack Problem*. As discussed by Han et al. (2012) [4], in online knapsack settings the scheduler faces a sequence of items (waves) arriving one by one.For each item, it must decide whether to "pack" it into the knapsack (e.g., send it over PCIe to the V100) or reject it (e.g., compute it locally on the H100). The "weight" of the item corresponds to its size in bytes (consuming PCIe bandwidth), and the "value" corresponds to the computational time saved by offloading.

The HeteFlow scheduler implements a greedy heuristic for this problem. By utilizing the "Wave" granularity insights from FlashOverlap, we ensure that each "item" placed in the knapsack is large enough to be bandwidth-efficient. This theoretical mapping suggests that the scheduler is not merely reacting to congestion heuristically, but is approximating a principled packing policy that aims to maximize the utilization of the slower legacy hardware without bottlenecking the critical path on the primary accelerators.

3 Research Plan

The core hypothesis of this research is that many inefficiencies in training Mixture-of-Experts (MoE) models on heterogeneous hardware arise not only from hardware limitations, but also from the rigidity of current scheduling policies. To investigate this hypothesis, we propose **HeteFlow**, a runtime system that replaces static workload assignment with dynamic, cost-aware scheduling. Our research plan is structured into three phases: theoretical formulation, system design and implementation, and empirical validation.

3.1 Theoretical Formulation: The Online Stochastic Scheduling Model

We formulate the MoE token routing process as an **Online Stochastic Resource-Constrained Project Scheduling Problem (SRCPSP)**. Unlike classical offline scheduling, where the entire task graph is known *a priori*, the MoE gating network generates workload stochastically at runtime. The system must make routing decisions for large numbers of tokens in short time intervals, with only partial information about future arrivals.

System Model. We define the cluster as a set of heterogeneous processing units $M = M_{fast} \cup M_{slow}$.

- M_{fast} represents modern accelerators (e.g., Nvidia H100s) characterized by high compute throughput μ_{fast} and high-bandwidth intra-node interconnects (e.g., NVLink) with bandwidth B_{nv}.
- M_{slow} represents legacy accelerators (e.g., Nvidia V100s) characterized by lower compute throughput μ_{slow} and more constrained inter-node interconnects (e.g., PCIe) with bandwidth B_{pci}.

Let the workload at timestep t be a set of token "Waves" $W_t = \{w_1, w_2, ..., w_n\}$. A wave is a contiguous block of tokens (e.g., 1024 tokens) targeting a specific Expert E_k. The duration of processing a wave w_i on machine m is a random variable $D_{i,m}$ with expected value $\mathbb{E}[D_{i,m}] = \text{Ops}(w_i)/\mu_m$.

Cost Function Derivation. The objective is to minimize the makespan of the current batch, which is typically dominated by the completion time of the slowest processor (the "straggler"). For a given wave w_i targeting an expert residing on a slow GPU (M_{slow}), the scheduler faces a binary decision: *Dispatch* or *Offload*.

Case 1: Dispatch (Standard Path). If the wave is sent to its home expert on M_{slow}, the cost $C_{dispatch}$ is the sum of transmission latency, queue wait time, and execution time:

$$C_{dispatch}(w_i) = \underbrace{\frac{Size(w_i)}{B_{pci}}}_{\text{Transmission}} + \underbrace{\text{Wait}(Q_{slow})}_{\text{Queuing Delay}} + \underbrace{\frac{\text{Ops}(w_i)}{\mu_{slow}}}_{\text{Execution}} \tag{1}$$

Here, $\text{Wait}(Q_{slow})$ is critical. As the workload skews toward the expert on M_{slow}, the queue length Q_{slow} grows. According to Little's Law, the wait time increases linearly with queue length, eventually causing the M_{slow} term to dominate the global batch time.

Case 2: Offload (Shadow Execution). Alternatively, a fast GPU M_{fast} can "steal" the wave and process it locally using a cached copy of the expert weights. The cost $C_{offload}$ involves loading the weights from memory and computing at high speed:

$$C_{offload}(w_i) = \underbrace{\frac{Size(E_k)}{B_{mem}} \cdot \mathbb{I}(E_k \notin \text{Cache})}_{\text{Weight Streaming}} + \underbrace{\frac{\text{Ops}(w_i)}{\mu_{fast}}}_{\text{Execution}} + \underbrace{\text{Penalty}_{H100}}_{\text{Opportunity Cost}} \tag{2}$$

The term $Size(E_k)/B_{mem}$ represents the cost of streaming the expert weights from CPU RAM or spare HBM if they are not already in the L2 cache. The Penalty_{H100} term accounts for the opportunity cost: while the H100 is processing the stolen wave, it is not processing its own assigned experts.

The Greedy ϵ-Lookahead Policy. Solving the full SRCPSP optimally is NP-hard. We therefore propose a lightweight greedy heuristic suitable for GPU kernel implementation. The scheduler maintains a running estimate of the queue depth Q_{slow} for each V100.

1. **Monitor:** At step t, the Gating Kernel outputs the routing histogram.
2. **Evaluate:** For each target expert E_k on a V100, we calculate the *Straggler Ratio* $S_k = \frac{C_{dispatch}}{C_{offload}}$.
3. **Decide:** If $S_k > \tau$ (where τ is a tunable threshold, typically 1.0), the V100 is deemed a bottleneck. The scheduler identifies the "overflow" waves contributing to this bottleneck.
4. **Action:** These overflow waves are flagged for local execution. The H100 triggers a high-priority DMA fetch for the weights of E_k (if not cached) and executes the waves using dedicated compute warps.

This policy can be interpreted as approximating an **Online Knapsack Solver**, where the "Knapsack" is the PCIe bandwidth and V100 compute capacity, and the "Items" are token waves. By rejecting items that "fit" too poorly (i.e., have high completion costs on V100), the scheduler aims to improve global throughput.

3.2 Engineering Methodology

TTranslating the theoretical model into a performant system requires addressing several engineering challenges. We leverage and extend recent systems and kernel primitives to build **HeteFlow**.

3.2.1 Granularity: The "Wave" Abstraction

A major challenge for applying prior systems like Comet to heterogeneous hardware is the "Granularity Mismatch." For example, Comet schedules work at the granularity of individual tiles (e.g., 128×128 tokens). On an NVLink-connected homogeneous cluster, the latency of sending a tile is negligible ($< 5\mu s$). However, on a heterogeneous cluster connected via PCIe Gen4, the launch overhead and protocol latency for very small messages can be significant.

Drawing on the analysis in *FlashOverlap* [6], we observe that PCIe bandwidth utilization tends to increase with message size and can degrade substantially below a certain threshold (e.g., around 256KB in their experiments). Therefore, HeteFlow introduces the **"Wave"** as the atomic unit of scheduling.

- **Definition:** A Wave is a collection of tokens (typically $512 \sim 2048$) grouped by their target expert.
- **Reordering:** Simply grouping tokens is insufficient if they are scattered in memory. We utilize the *Pre-Communication Reordering* kernel from FlashOverlap. This kernel fuses the reordering operation into the Gating phase, physically moving tokens into contiguous memory buffers.
- **Impact:** This ensures that every PCIe transaction is a bulk transfer, saturating the bus bandwidth. It also simplifies the scheduler's state space: instead of making per-token decisions at very high frequencies, it makes decisions per wave, reducing CPU/GPU control overhead.

3.2.2 Communication Primitives: The ParallelKittens Framework

Implementing a kernel that simultaneously computes (local experts), communicates (PCIe dispatch), and manages a dynamic schedule is prohibitively complex in raw CUDA. To manage this complexity, we utilize the **ParallelKittens (PK)** [13] framework, which provides C++ templates for managing the GPU memory hierarchy. Specifically, we extend PK's *Load-Compute-Store-Communicate (LCSC)* template to implement a **Polymorphic Dispatcher**.

In a standard kernel, all warps (groups of 32 threads) perform the same instruction. In our Polymorphic Dispatcher, we partition the Streaming Multiprocessor (SM) resources on the H100:

- **Warps 0-3 (The Compute Engine):** These warps are dedicated to tensor core math. In the default state, they load local expert weights and process local tokens. In the "Shadow Execution" state, they receive a signal from the scheduler to load a remote expert's weights (via TMA) and process offloaded waves.

- **Warps 4-7 (The Communicator):** These warps never perform math. They are dedicated to driving the PCIe interconnect. They manage the circular ring buffers used to stream waves to the V100s. Crucially, they also execute the *Scheduler Logic*. By checking the occupancy of the ring buffer (which proxies for the V100's queue depth), they make the real-time decision to signal Warps 0-3 to take over "Shadow" work.

This separation of concerns, enabled by PK's warp specialization primitives, allows computation and communication to proceed largely asynchronously, helping to reduce pipeline stalls.

3.2.3 Shadow Expert Management and Memory Hierarchy

The "Shadow Expert" mechanism relies on the H100 having timely access to the V100's expert weights. Since HBM capacity is limited (e.g., 80GB on an H100), we cannot simply replicate the entire MoE model on every GPU. We implement a **Three-Tier Memory Hierarchy**:

1. **Tier 1 (H100 L2 Cache / HBM):** Stores the "Home" experts assigned to this GPU.
2. **Tier 2 (Pinned CPU RAM):** Acts as a "Hot Cache" for the V100's experts. We use a Least-Recently-Used (LRU) policy to keep the weights of the most frequently activated V100 experts in the host's pinned memory.
3. **Tier 3 (V100 HBM):** The authoritative copy of the V100's experts.

When the scheduler decides to offload a wave to the H100, it must fetch the weights. If the weights are in Tier 2, the H100 initiates a host-to-device transfer over PCIe Gen5 (which is significantly faster than the peer-to-peer link to the V100). To hide this fetch latency, we implement **Double Buffering**: the Compute Warps process Wave N (local) while the TMA pre-fetches the weights for Wave $N + 1$ (shadow) into Shared Memory.

3.3 Implementation and Evaluation Plan

Implementation Strategy. We plan to implement HeteFlow as a custom PyTorch extension. The core runtime will be written in C++ / CUDA using the ParallelKittens template library.

- **Month 1:** Reproduction of baselines. We will set up the HeterMoE (DeepSpeed-based) and Comet environments and benchmark them to establish baseline performance and limitations.
- **Month 2:** Kernel development. We will implement the Polymorphic Dispatcher using ParallelKittens, including specialized warp logic for the H100 and a simplified "worker" kernel for the V100.
- **Month 3:** Scheduler Integration. We will fuse the Greedy ϵ-Lookahead logic into the Gating kernel. This phase requires tuning the threshold τ to balance the cost of weight streaming vs. the cost of waiting.
- **Month 4:** Full System Integration and Evaluation.

Evaluation Setup. We plan to conduct experiments on a representative heterogeneous cluster comprising 2 nodes. Node A contains 2$\times$ Nvidia H800 GPUs (representing the "Fast" tier), and Node B contains 4$\times$ Nvidia V100 GPUs (representing the "Slow" tier). This 1:2 ratio roughly mimics scenarios where older hardware is more abundant but less capable.

Workloads. We will evaluate HeteFlow using the **Mixtral-8x7B** architecture [9], a representative sparse MoE model.

- **Dataset A (Standard):** The C4 (Colossal Clean Crawled Corpus) dataset. This represents "natural" skew inherent in language.
- **Dataset B (Stress Test):** A synthetic dataset generated to follow a Zipfian distribution with $\alpha = 1.2$. This setting concentrates a large fraction of tokens on a small subset of experts (e.g., 80% of tokens to 10% of experts).

Baselines. We compare HeteFlow against three distinct systems:

1. **HeterMoE (Vanilla):** This represents a recent approach for heterogeneous MoE training. It uses static Zebra Parallelism. We hypothesize it will perform well on Dataset A but collapse on Dataset B due to straggler effects.
2. **Comet (Vanilla):** This represents a recent state-of-the-art approach for homogeneous fine-grained overlapping. We use it to examine whether fine granularity *alone* is sufficient without skew-aware scheduling; in heterogeneous settings, slower V100s may cause pipeline bubbles that Comet cannot fully hide.
3. **DeepEP (Manual):** We will implement a baseline using DeepSeek's optimized communication library but without our dynamic offloading logic. This serves as an ablation study to help isolate the benefits of the "Shadow Expert" mechanism from the benefits of pure communication optimization.

Metrics.

- **End-to-End Throughput:** Measured in tokens per second. This is the primary figure of merit.
- **Bubble Rate:** We will use Nsight Systems to profile the H100 Tensor Core utilization. We aim to reduce the "Idle" time (caused by waiting for V100s) from an expected $\sim 30\%$ in HeterMoE to $< 5\%$ in HeteFlow.
- **PCIe Efficiency:** We will measure effective bus utilization to evaluate whether the "wave" granularity achieves higher link utilization compared to Comet's tile-based transfers.
- **Convergence:** Finally, we will verify that our dynamic rebalancing does not harm model convergence. Since HeteFlow only changes *where* computation happens, not *what* computation happens, the loss curve should be identical to standard training, confirming numerical correctness.

We hypothesize that HeteFlow can achieve a 1.2–1.5$\times$ speedup over HeterMoE in high-skew scenarios, effectively reducing the performance impact of hardware heterogeneity and enabling more efficient training of large MoE models under skewed workloads.

References

[1] R. D. Blumofe and C. E. Leiserson. Scheduling multithreaded computations by work stealing. *Journal of the ACM (JACM)*, 46(5):720–748, 1999.

[2] DeepSeek-AI. DeepEP: Efficient expert-parallel communication library. `https://github.com/deepseek-ai/DeepEP`, 2025.

[3] T. Gale, D. Narayanan, C. Young, and M. Zaharia. Megablocks: Efficient sparse training with mixture-of-experts. In *Proceedings of Machine Learning and Systems*, volume 5, pages 288–304, 2022.

[4] X. Han, Y. Kawase, and K. Makino. Online knapsack problem with removal cost. In J. Gudmundsson, J. Mestre, and T. Viglas, editors, *Computing and Combinatorics*, pages 61–73, Berlin, Heidelberg, 2012. Springer Berlin Heidelberg.

[5] J. He, J. Qiu, A. Zeng, Z. Yang, J. Zhai, and J. Tang. Fastmoe: A fast mixture-of-expert training system. *arXiv preprint arXiv:2103.13262*, 2021.

[6] K. Hong, X. Li, M. Liu, Q. Mao, T. Wu, Z. Huang, L. Chen, Z. Wang, Y. Zhang, Z. Zhu, et al. Efficient and adaptable overlapping for computation and communication via signaling and reordering. *arXiv preprint arXiv:2504.19519*, 2025. Accepted to EuroSys '26.

[7] C. Hwang, W. Cui, Y. Xiong, Z. Yang, Z. Liu, H. Hu, Z. Wang, R. Salas, J. Jose, P. Ram, et al. Tutel: Adaptive mixture-of-experts at scale. In *Proceedings of Machine Learning and Systems*, volume 5, pages 269–287, 2022.

[8] R. A. Jacobs, M. I. Jordan, S. J. Nowlan, and G. E. Hinton. Adaptive mixtures of local experts. *Neural computation*, 3(1):79–87, 1991.

[9] A. Q. Jiang, A. Sablayrolles, A. Roux, A. Mensch, B. Savary, C. Bamford, D. S. Chaplot, D. de Las Casas, E. B. Hanna, F. Bressand, et al. Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024.

[10] D. Lepikhin, H. Lee, Y. Xu, D. Chen, O. Firat, Y. Huang, M. Krikun, N. Shazeer, and Z. Chen. Gshard: Scaling giant models with conditional computation and automatic sharding. *arXiv preprint arXiv:2006.16668*, 2020.

[11] M. L. Pinedo. *Scheduling: theory, algorithms, and systems*. Springer, 2016.

[12] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538*, 2017.

[13] S. H. Sul, S. Arora, B. F. Spector, and C. Ré. ParallelKittens: Systematic and practical simplification of Multi-GPU AI kernels. *arXiv preprint arXiv:2511.13940*, 2025.

[14] H. Topcuoglu, S. Hariri, and M.-Y. Wu. Performance-effective and low-complexity task scheduling for heterogeneous computing. *IEEE transactions on parallel and distributed systems*, 13(3):260–274, 2002.

[15] Y. Wu, X. Liu, S. Jin, C. Xu, F. Qian, Z. M. Mao, M. Lentz, D. Zhuo, and I. Stoica. HeterMoE: Efficient training of mixture-of-experts models on heterogeneous GPUs. *arXiv preprint arXiv:2504.03871*, 2025.

[16] S. Zhang, N. Zheng, H. Lin, Z. Jiang, W. Bao, C. Jiang, Q. Hou, W. Cui, S. Zheng, L.-W. Chang, et al. COMET: Fine-grained computation-communication overlapping for mixture-of-experts. *arXiv preprint arXiv:2502.19811*, 2025.